First, we need to reject users who already received a payment this month. Let's say you dispatch this job, but there were some errors with 3 users. You should be able to re-run it without any side effects. This check avoids paying the same user twice.

I created a small "helper" class called `DateService`:

```
namespace App\Services;

class DateService
{
  public static function lastMonth(): DateInterval
  {
    return DateInterval::make(
      startDate: StartDate::fromCarbon(now()->subMonthNoOverflow()->startOfMonth()),
      endDate: EndDate::fromCarbon(now()->subMonthNoOverflow()->endOfMonth()),
    );
  }

  public static function thisMonth(): DateInterval
  {
    return DateInterval::make(
      startDate: StartDate::fromCarbon(now()->startOfMonth()),
      endDate: EndDate::fromCarbon(now()->endOfMonth()),
    );
  }
}
```

The responsibility of this class is to encapsulate and generalize some basic date-related functions. Later, we'll extend it with more methods.